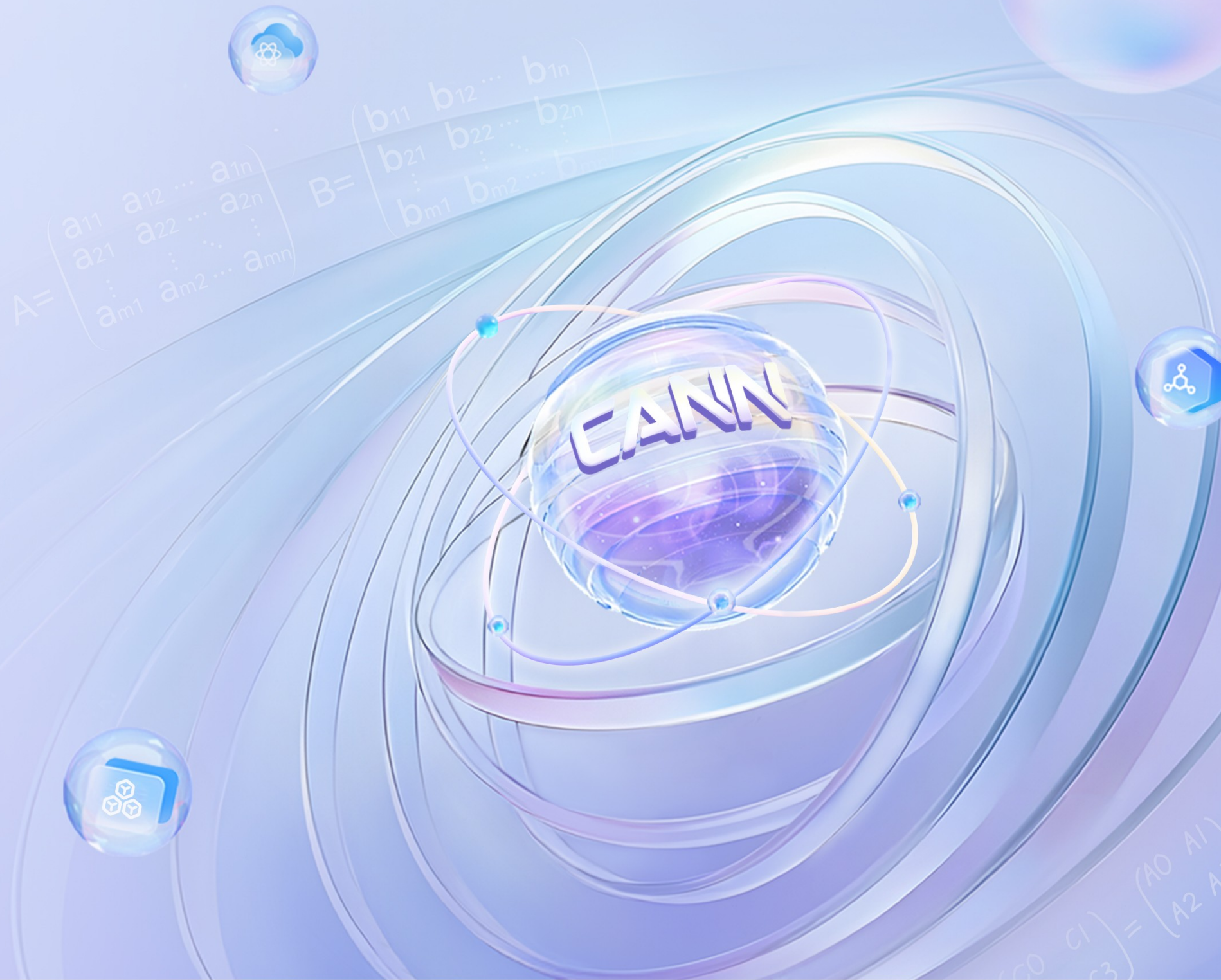


大模型推理优化基础 从原理到实践

昇腾 CANN 训练营

融合算子 · 图模式 · KV Cache · 并行



目录

- 1 推理为什么需要优化
- 2 融合算子与 AscendC
- 3 图模式优化
- 4 KV Cache 优化
- 5 多卡并行优化

01 推理为什么需要优化

从三大开销来源理解推理优化的方向



推理优化

这门课只围绕一个问题展开：当若干用户请求进入大模型服务后，系统要如何用更低成本、更低时延、更高吞吐地生成下一个 token？

单卡推理优化

多卡不是魔法，如果单卡性能低下的问题，多卡只会把问题放大

单卡上数据通路一般为：

将权重 / 输入 / KV cache 从 HBM 搬入核内 buffer

执行 Attention/MLP/Norm 等计算

将结果搬出 HBM

优化思路：少搬、快算、边搬边算、少写中间结果

量化：少搬、快算

搬运计算掩盖：边搬边算

融合算子：少写中间结果

图优化：整个模型执行流程的优化

KVCache 优化：少搬

多卡推理优化

当模型单卡放不下，或吞吐不足时，就要进入多卡部署

多卡会带来通信，是影响多卡性能线性扩展的关键原因

不同的切分策略也会带来不同的性能表现

优化思路

多卡切分策略设计

通信隐藏的多流并行

TTFT（Time To First Token）：首 token 时延

TPOT（Time Per Output Token）：单个输出 token 的时延

吞吐（Throughput）：系统单位时间为所有请求生成的 token 总数，
单位 token/s

02 融合算子与 AscendC

若干基础操作的融合，搬运与计算的流水线并行



融合算子：把多个运算合并成一个

计算公式： $y = \text{RMSNorm}(x + \text{residual})$ 其中 $\text{RMSNorm}(z) = \frac{z}{\sqrt{\text{mean}(z^2) + \epsilon}} \times \text{weight}$

不融合

Add → GM → Square
→ GM → Mean
→ GM → Sqrt
→ GM → Div
→ GM → Mul(weight)
→ GM

6 个 kernel
每步都经过 Global Memory

RMSNorm 融合

Add → Global Memory
→ npu_rms_norm
(Square→Mean→Sqrt→Div→Mul
合并为一个 kernel)

2 个 kernel
RMSNorm 内部操作已融合
但仍需 Add 后写回、再读出

Add+RMSNorm 完全融合

npu_add_rms_norm
(Add + RMSNorm 全部
合并为一个 kernel)

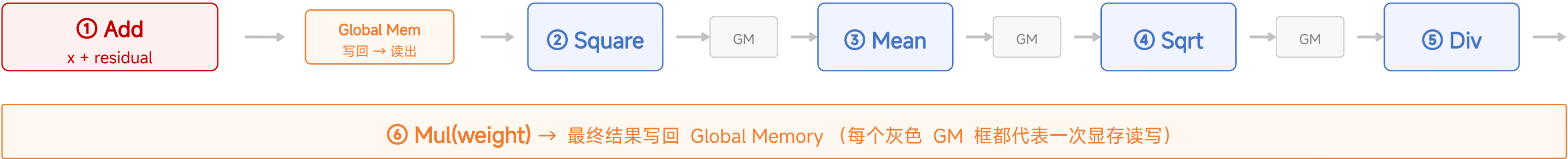
1 个 kernel
中间结果全程留在片上
只需 1 次 Global Memory 往返

融合的深度可以不同：RMSNorm 本身已经是融合算子（把 5 个小操作合并），但 Add 和 RMSNorm 之间还可以进一步融合，消除两者之间的 Global Memory 往返。Qwen3-8B 在 CANN 上使用的是 npu_add_rms_norm。



不融合时：数据在显存和计算单元之间反复搬运

把 RMSNorm 展开为最基本的操作，看一次 Add+RMSNorm 涉及多少数据搬运



RMSNorm 的计算拆解

$$\text{RMSNorm}(z) = z / \sqrt{(\text{mean}(z^2) + \epsilon) \times \text{weight}}$$

- ① Square : z^2 (逐元素平方)
- ② Mean : 对 z^2 求均值
- ③ Sqrt : 均值加 ϵ 后开方 (得 RMS)
- ④ Div : z 除以 RMS (归一化)
- ⑤ Mul : 乘以可学习的 weight

统计

Kernel launch : 6 次
Global Memory 读写: 6 次以上

瓶颈

每个小操作都要读写显存
计算被数据搬运掩盖

在实际的 CANN 实现中，RMSNorm 的 5 个小操作已经被融合为一个 kernel。但 Add 和 RMSNorm 之间仍然需要经过 Global Memory——这就是进一步融合的机会。



融合后：中间结果留在片上，只写回一次

Add + RMSNorm 融合为 npu_add_rms_norm，数据在片上缓存中完成全部计算

npu_add_rms_norm(residual, x, weight, eps)

Add + Square + Mean + Sqrt + Div + Mul 在一个 kernel 内完成

融合后的数据流动

读取输入
x, residual, weight



Unified Buffer (片上)
Add → Square → Mean →
Sqrt → Div → Mul 连续计算



写回结果
y → Global Memory

只需 1 次
Global Memory 往返

不融合： 6 个 kernel，6 次以上 Global Memory 往返

融合后： 1 个 kernel (npu_add_rms_norm)，1 次 Global Memory 往返，中间结果全程留在片上

AscendC：为昇腾 NPU 定制算子

AscendC 是什么

昇腾 AI 处理器上的 C/C++ 算子开发语言

可以直接描述：

- 数据搬运（Global Memory ↔ 片上缓存）
- 计算流水（多指令并行执行）
- 片上存储（UB/L1 的精细管理）

为什么用 AscendC

通用框架算子无法充分利用 NPU 硬件特性

AscendC 可以：

- 控制 **tiling**：把大数据切分成片上能容纳的小块
- 实现 **流水并行**：搬运与计算重叠
- 复用 **片上缓存**：减少 Global Memory 访问

AscendC 编程的核心概念



这三个概念共同决定了融合算子的效率：tiling 决定数据怎么切，流水并行决定指令怎么排，多核协同决定任务怎么分。

AscendC 让开发者可以像写 C++ 一样写 NPU 算子，同时精细控制硬件资源，是实现高效融合算子的关键工具。

融合算子实例： Attention 融合

不融合的 Attention 步骤

1. $Q \times K^T \rightarrow$ 得到 Attention Score
2. Scale (除以 $\sqrt{d_k}$)
3. Softmax (归一化)
4. Apply Mask (可选)
5. Score $\times V \rightarrow$ 得到输出

问题：每步都读写 Global Memory

5 个 kernel，多次显存往返

融合后的 Attention

FusedAttention：一个 kernel 完成

$Q, K, V \rightarrow$ 片上计算 $\rightarrow$ 输出

收益：中间结果不写出，显存带宽压力大幅降低

API: `torch_npu.npu_fused_infer_attention_score`

这是 CANN 提供的融合 Attention 算子，推理时可直接调用。

融合算子的核心价值：在推理框架中，融合算子已经成为标配优化手段。CANN 提供了大量预置融合算子（如 FusedAttention、FusedMatMul 等），开发者也可以基于 AscendC 自定义融合算子来进一步优化特定场景。

对于 Qwen3-8B 这样的小模型，在单卡 NPU 上使用融合算子是提升推理效率的**首要手段**。

03 图模式优化

模型的系统性调度，让执行更高效



从逐算子执行到图模式

Eager 模式：逐算子下发

执行流程：

Python → 框架 → 算子 1 下发 → NPU 执行
→ 算子 2 下发 → NPU 执行
→ ... （每步都要层层调度）

开销来源：

- Python → C++ 层切换
- 框架图优化、内存分配
- 驱动 → NPU 指令下发
- 每步都有同步等待

图模式：编译后统一执行

执行流程：

Python → **图捕获 / 编译** → 统一调度
→ NPU 连续执行全部算子

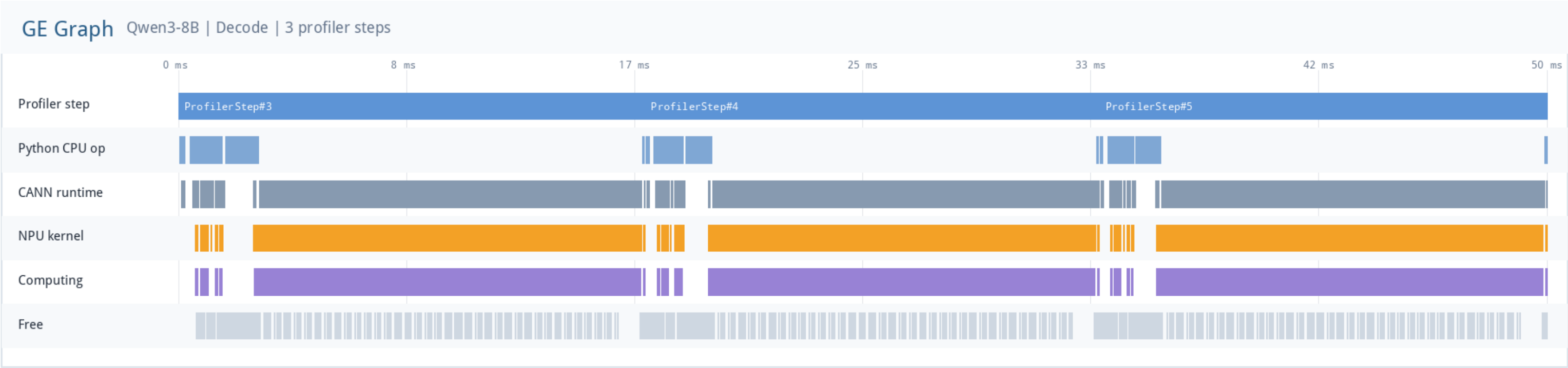
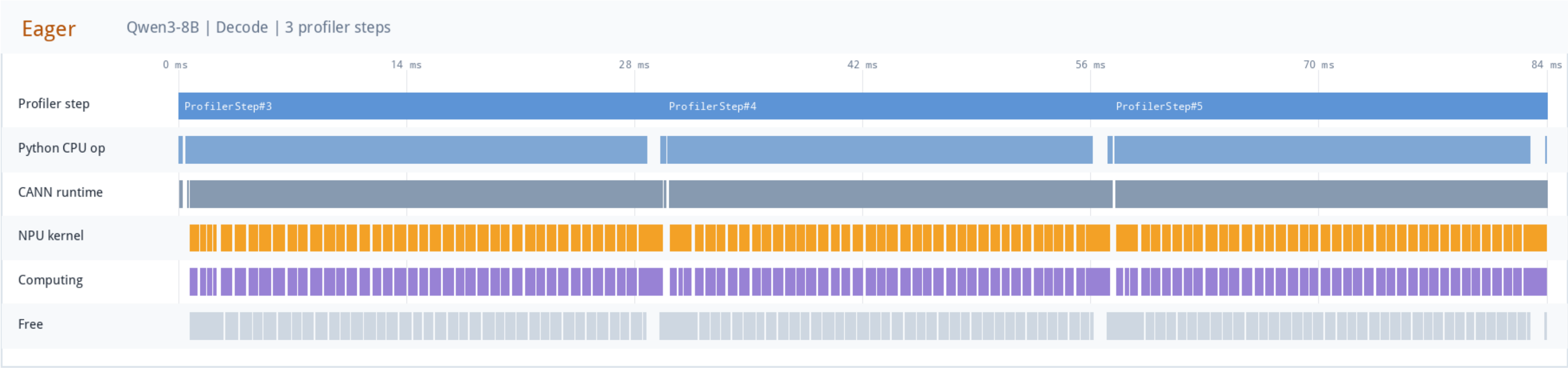
收益：

- 一次编译，多次复用
- 减少 Python/ 框架调度
- 便于算子融合、内存规划
- 执行计划更稳定可预测

不改变数学结果，改变执行方式

图模式的核心思想是 "**先编译，后执行**"。在推理场景中，模型结构和执行路径相对固定，图模式可以把编译开销摊平到多次执行中，显著降低平均延迟。

Qwen3-8B Decode 阶段 Eager 模式和 GE 图模式对比



NPU 上的图模式实践

ge_graph : 静态图编译

- 模型加载时一次性编译整个计算图
- 编译后进行算子融合、内存规划、并行编排
- 适合固定 shape、稳定执行路径的场景
- 首次编译有开销，后续执行高效稳定

npugraph_ex : 图捕获复用

- 首次执行时捕获计算图并缓存
- 后续相同输入 shape 直接复用缓存
- 适合 Decode 小步循环等高频重复场景
- 更灵活，支持动态 shape 变化

图模式的使用流程



工程建议： 在线服务启动后用典型输入做预热，确保图编译完成并写入缓存，避免首批请求遇到冷启动延迟。

04 KV Cache 优化

长上下文下的困境与解决



KV Cache 的管理与压缩

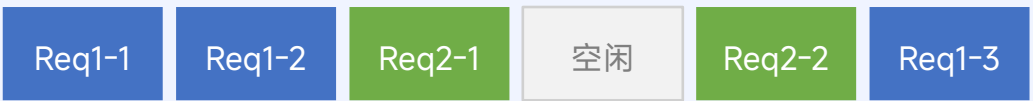
Paged Attention : 分页管理

问题：传统 KV Cache 预分配连续空间，短请求浪费、长请求不足。

方案：切成固定大小的 block/page :

- 逻辑连续 → 物理不连续块映射
- 按需分配，用完释放

物理内存中的块分配



■ 请求 1 ■ 请求 2 ■ 未分配

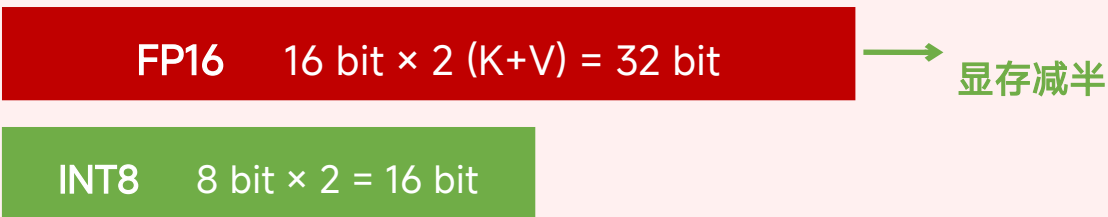
KV Cache 量化压缩

问题：长上下文下 KV Cache 占用大量显存。

方案：降低 KV 存储精度：

- FP16 → INT8 : 显存减半，精度可控
- 部分层量化：只压缩不关键层

精度对比



权衡：压缩越激进显存省越多，但可能影响长文本精度

KV Cache 优化的核心目标：在保持推理质量的前提下，用更少的显存管理更多的历史状态。

Paged Attention 解决 "怎么管" 的问题，量化压缩解决 "占多少" 的问题，两者可以结合使用。

05 多卡并行优化

模型太大或吞吐不够时的有力措施

多卡并行策略

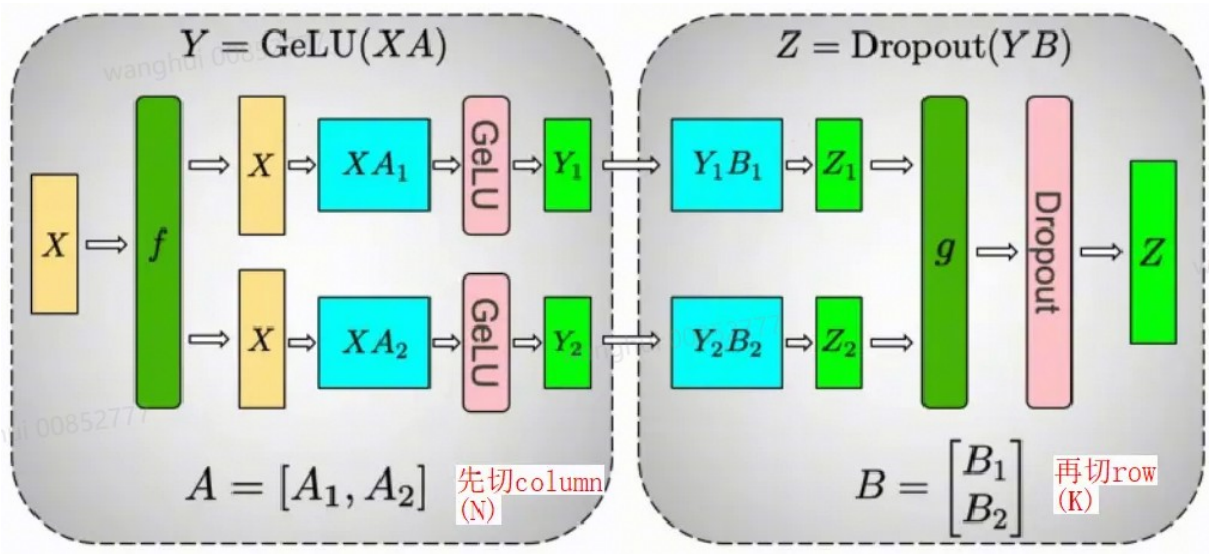
在具有上亿级别参数的大模型推理的过程中，由于模型的权重过大或者是输入数据过大导致无法加载到一张卡上或一台机器上，此时就需要将推理计算任务分布到多张卡或多个计算机上，当前常用的分布式并行策略有数据并行（Data Parallelism），张量并行（Tensor Parallelism，TP），流水线并行（Pipeline Parallelism）等。

数据并行（DP）：把输入数据按 Batch 轴切成 N 份，并行输入给模型推理，每张卡的模型一致。输出结果再拼接，得到整网的输出。

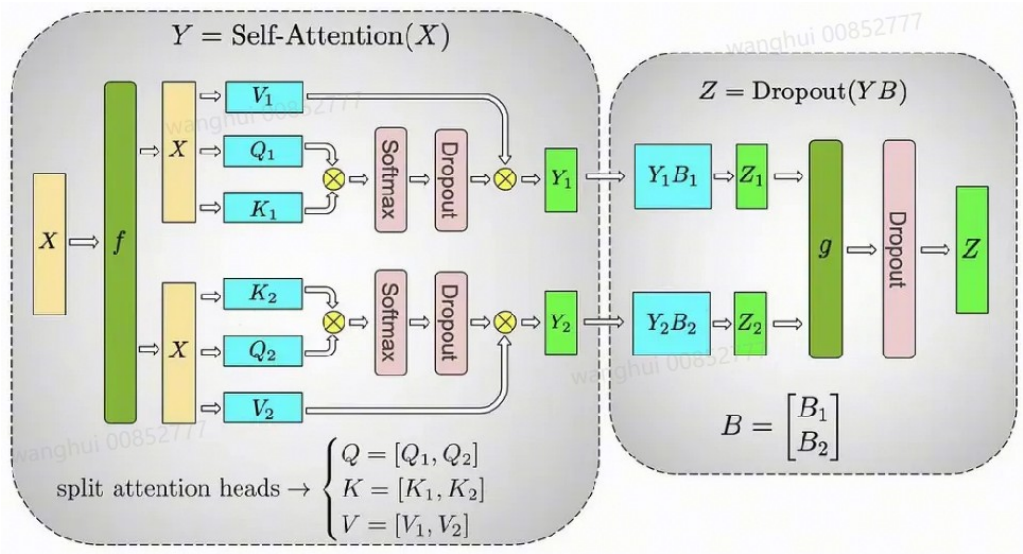
序列并行（SP）：把输入数据按 Sequence 轴切成 N 份，可以对单 Batch 的输入数据进行切分。

张量并行（TP）：切分权重，将矩阵计算切分到多张卡上，并行计算。将权重 shape 记为 [K, N]，可分为切 K 和切 N 两种方式。

专家并行（EP）：MoE 网络中，将专家切分到不同卡上，实现专家间的计算并行。



TP 切分 MLP，Matmul1 切 N，Matmul 切 K



TP 切分 Attention，QKV 切 head，Matmul2 切 K

矩阵乘法：单卡上的完整计算

大模型中的线性层本质上都是矩阵乘法 $Y = X \times W$ 。先看单卡上如何完成这个计算。

$$Y = X \times W$$

其中 X : [batch, d_in] , W : [d_in, d_out] , Y : [batch, d_out]



单卡执行： X 和完整的 W 都在同一张卡上，一次矩阵乘法得到 Y 。当 W 太大（如 $d_{in}=4096, d_{out}=4096$ ）单卡放不下或计算太慢时，就需要把 W 切分到多张卡上。

矩阵切分：大任务拆成小任务

矩阵乘法天然支持分块计算。切分后每卡算一块，最后汇总——结果等价，但存储和计算都均摊了

方式一：W 按列切分（X 不变）

X
完整

×

W₀
卡 0

W₁
卡 1

=

Y₀
卡 0

Y₁
卡 1

→ 拼接
[Y₀|Y₁] = Y

W 按列切成 W₀, W₁，X 不变广播到所有卡。每卡计算 $Y_i = X \times W_i$ ，最后按列拼接得到完整 Y。每张卡只存 1/2 权重。

方式二：X 按列切，W 按行切

X₀
卡 0

X₁
卡 1

×

W₀
卡 0

W₁
卡 1

=

Y₀
卡 0

Y₁
卡 1

→ 相加
Y₀+Y₁=Y

X 按列切成 X₀, X₁（左右两块），W 按行切成 W₀, W₁（上下两块）。每卡计算 $Y_i = X_i \times W_i$ ，最后相加得到 Y。每张卡只处理 1/2 数据。

核心结论：无论哪种切分方式，最终结果与单卡计算**完全等价**。但每张卡只存部分权重、只算部分数据，存储和计算量均摊到多张卡。这就是多卡并行的基本原理：大矩阵拆成小矩阵，多卡协同，结果不变，压力均摊。

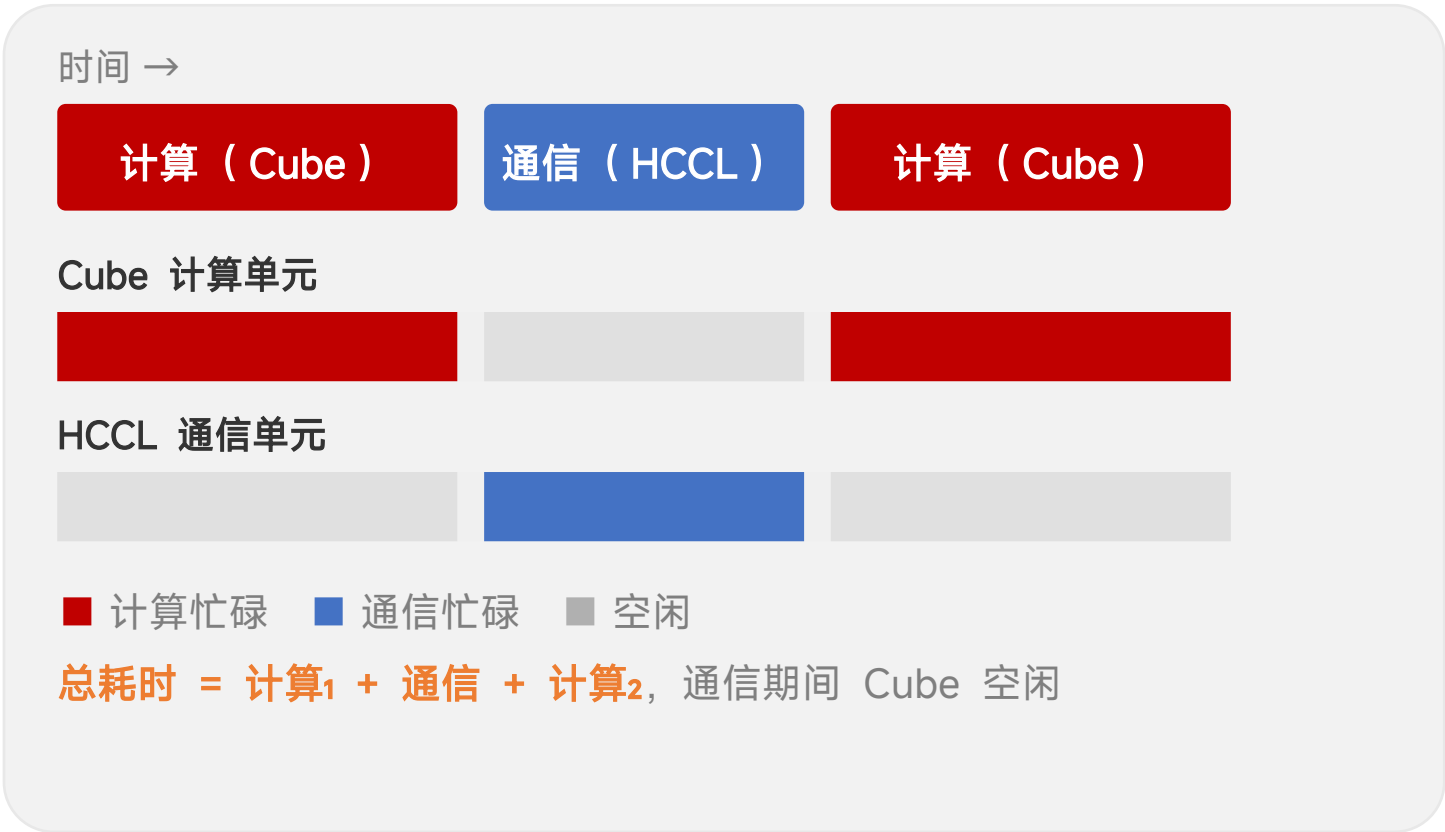
什么时候需要多卡？ 单卡放不下模型权重、batch 太大一张卡算不完、需要更低延迟。并行不是让单卡更快，是让更多卡一起干活。



多流并行：计算与通信重叠

NPU 上有不同硬件单元（Cube 计算、HCCL 通信），多流让它们同时工作，缩短总耗时

单流：计算与通信串行



多流：不同硬件单元并行工作



多流并行是 CANN 提供的重要优化手段。NPU 上的 Cube（计算）和 HCCL（通信）是独立的硬件单元。单流时它们串行工作，多流时它们**并行工作**——通信不再阻塞计算，从而在不增加算力的前提下缩短关键路径，提升 NPU 利用率。



推理优化技术地图

1 融合算子

多个运算合并为一个 kernel

解决：算子调度与中间结果读写开销

工具：AscendC

2 图模式

编译成计算图统一调度执行

解决：Python/ 框架逐算子调度开销

工具：ge_graph / npugraph_ex

3 KV Cache 优化

分页管理 + 量化压缩

解决：显存占用过高、并发容量不足

手段：Paged Attention / INT8

4 多卡并行

矩阵切分 + 多卡协同

解决：单卡容量与算力不足

本质：大任务拆小任务

优化思路：先明确瓶颈在哪里——是算子效率低、调度开销大、显存不够放？再从对应的技术手段中选择合适的优化策略。

对于 Qwen3-8B + 单卡 NPU 场景，优先关注**融合算子**和**图模式**，收益最直接。

Thank you.

社区愿景：打造开放易用、技术领先的 AI 算力新生态

社区使命：使能开发者基于 CANN 社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的 CANN 生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上 CANN 社区获取干货



关注 CANN 公众号获取资讯